

Developer Onboarding

Everything you need to go from a fresh clone to your first change — and a guided path into the canonical docs so you know where every truth lives.

[~30 min read](#)[read top→bottom](#)[Next.js 16 · Supabase · Clerk](#)[updated 2026-07-14](#)

Read this first. This guide is a **map into** the project's canonical sources, not a replacement for them. It gives you the mental model, the setup, and a reading order — then points to the file that owns each detail (`CLAUDE.md`, `CONTEXT.md`, the `wiki/`, the Codebase Map). When this guide and a canonical source ever disagree, the source wins.

1. [01](#) What TVI-CAMS is (and what it is not)
2. [02](#) The five-minute mental model
3. [03](#) Get it running
4. [04](#) Architecture at a glance
5. [05](#) How the code is organized
6. [06](#) Your guided reading path
7. [07](#) Rules that will trip you up
8. [08](#) Workflow: Linear, branches, PRs
9. [09](#) Where things stand today
10. [A](#) Appendix: domain glossary

01 What TVI-CAMS is (and what it is not)

TVI-CAMS is an **internal multi-tenant compliance working layer** for TVI schools running TESDA scholarship batches (TWSP / CFSP). It tracks batch lifecycle, documents, attendance and LAMR evidence, and generates the TVI's own billing documents.

The boundary — internalize this. TESDA's SIS / T2MIS / BSRS remain the **authoritative** systems. This tool holds internal working copies only and **never** official standing. UI copy must never imply official approval or submission. This isn't a nicety — it's a correctness constraint on every screen and every string you write.

- **Tenants:** three initial schools — AKB, J3ED, NEN. Strict RLS scoping.
- **Lifecycle:** AOU → NTP → TIP → Training → Assessment → Billing.
- **Roles:** Admin, Coordinator, Trainer, Viewer.
- **Honesty by copy:** prototype screens portray the ADR-001 target even where schema isn't migrated (ADR-002).

Owns the detail: README.md · CONTEXT.md · docs/adr/ADR-002

02 The five-minute mental model

These terms are the project's ubiquitous language. Learn them and the rest of the codebase reads itself. The right column is the wording to **avoid** — using it signals the wrong model.

TERM	MEANS	AVOID
Batch	One RQM code's worth of scholars trained toward one qualification — the unit everything hangs off. One RQM code = one batch.	Class, cohort, group
RQM	The Request-for-QM authorization code naming a batch. Modeled as attributes <i>on the batch</i> (rqm_code, ntp_number, approved_slots...), never a separate table.	A PROGRAM_RQM table (rejected design)
NTP	Notice To Proceed — the approval authorizing a batch to run. Lives on the batch.	—
Approved slots	The billable-pax cap for a batch. You may bill for fewer scholars, never more.	—
ULI	The permanent national learner key (TESDA-issued). Same person at two schools = two tenant-scoped rows sharing one ULI.	Learner id, student number
Training progress	sessions_held ÷ total_sessions (total = nominal_hours ÷ 8), snapshotted on the batch. Attendance-derived, never time-elapsd.	% complete, days elapsed
Ineligible scholar	A scholar with ≥5 absences , excluded from the allowance. The only eligibility rule (stored as max_absences=4; copy always says "≥5").	attendance-% rule
Billing track	One of three parallel schedules a batch bills on — Training Cost, TSF/Allowance, Entrepreneurship — each on its own clock. Assessment Fee is out of scope.	"the billing" (there are three)
Billing tranche	A single releasable payment within a track, unlocked at an attendance threshold. Answers "which money is releasable," not "how far through training."	installment, milestone payment
Readiness gate	Per-tranche condition unlocking document generation: threshold reached AND that tranche's supporting docs all Verified. Tracked at tranche level, not per scholar.	"billing-ready" = threshold only
Billing document	A populated .docx generated from the school's own template. Append-only history — corrections re-generate a new versioned snapshot.	invoice, bill

Owns the detail: CONTEXT.md (full glossary, incl. the three Entrepreneurship concepts) · appendix of this guide

03 Get it running

Prerequisites: **Node.js** (version per Next.js 16's requirement) and **npm**. Then:

```
git clone <repo> → cd tesda-compliance-manager → npm install → cp .env.local.example .env.local → npm run dev
```

Environment (.env.local). The one gotcha that stops a naive setup:

VARIABLE	STATUS	NOTES
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY	Required	Clerk auth; middleware protects every route, so the app will not boot without it.
CLERK_SECRET_KEY	Required	Server-side Clerk key. Pair with the publishable key above.
NEXT_PUBLIC_SUPABASE_URL	Optional	Omit both → data layer returns unconfigured and the app runs on mock data (shared/mocks). Set both → live Supabase.
NEXT_PUBLIC_SUPABASE_ANON_KEY	Optional	
NEXT_PUBLIC_DEMO_EMAIL / _PASSWORD	Optional	Prefill for the demo sign-in. Not needed for development.

You can run with mock data. Configure **Clerk only** and leave Supabase unset — the data layer returns unconfigured and the app renders the seed dataset from `shared/mocks`. That is the fastest way to see every screen without provisioning a database. Clerk keys, however, are **not** optional: the middleware protects every route, so the app will not boot without them.

Commands

COMMAND	WHAT IT DOES
npm run dev	Next.js dev server (start here)
npm run build	production build
npm run lint	ESLint (flat config; static dirs globally ignored)
npx tsc --noEmit	typecheck (strict) — no dedicated script
npm run preview	serve the static design preview bundle on :5000
npm test	placeholder — no test runner yet (Phase 0.4)

There is no test runner yet — standing up Vitest + real-Supabase integration tests is Phase 0.4. The dashboard also accepts `?role= / ?state=` preview overrides until the real tenant/role resolver lands (TES-34).
Owns the detail: `CLAUDE.md (Commands) · package.json · lib/supabase/server.ts`

04 Architecture at a glance

A single **Next.js 16 App Router** app (React 19, TS strict, Tailwind v4) talking **directly to Supabase** (Postgres + Storage), with **Clerk** as identity. No separate backend. Every request flows through this chain:

Browser

client islands: sidebar, modals, uploads



Clerk middleware — proxy.ts

protects every route except /sign-in, /sign-up



Next.js Server Component

fetch + role-scoped render; calls a module data/ layer



createSupabaseServerClient()

attaches Clerk JWT (template "supabase") as bearer on an anon-key client



Postgres + RLS (app_private.*)

the security boundary — every authorization decision; a wrong query returns zero rows

The one load-bearing idea: RLS is the security boundary; hiding things in the UI is usability only. Authorization is decided in Postgres, so even a wrong query returns zero unauthorized rows. Never filter by tenant in JS — RLS already scopes the rows.

A dedicated backend (**Express.js**, Node/TS) is documented as future-only — do not build it or treat it as present. (The recommendation changed from Laravel on 2026-07-06.)

Owens the detail: [wiki/Architecture Overview](#) · [wiki/Auth And Tenancy](#) · [docs/TRD.md](#)

05 How the code is organized

Code is grouped by **domain, not file type** (the TES-68 DDD layout). Import direction is ESLint-enforced:

```
app/ → modules/<domain>/ {data, domain, ui} → shared/ → lib/supabase
```

- **app/** — thin routes; Server Components fetch + compose. No business logic.
- **data/** — **fetch → map → derive**; the only layer that may import `database.types`.
- **domain/** — pure rules, no I/O; unit-testable with fixed as-of dates.
- **ui/** — domain-aware components; reuse `shared/ui` primitives, no barrels, deep imports.

Two type families, kept apart: `lib/supabase/database.types.ts` (generated raw rows) vs `shared/types.ts` (UI domain types). Components see domain types only. The reference implementation to copy is `modules/batches/data/batches.ts`.

Owns the detail: `CLAUDE.md (Architecture)` · `wiki/Data Layer Pattern` · `Codebase Map.md` + `codebase/*.md`

The vault trick: this repo *is* an Obsidian vault. Open the **graph view** to see the code cluster (one node per module) fused to the concept cluster (wiki notes). The map below is that same structure on paper.

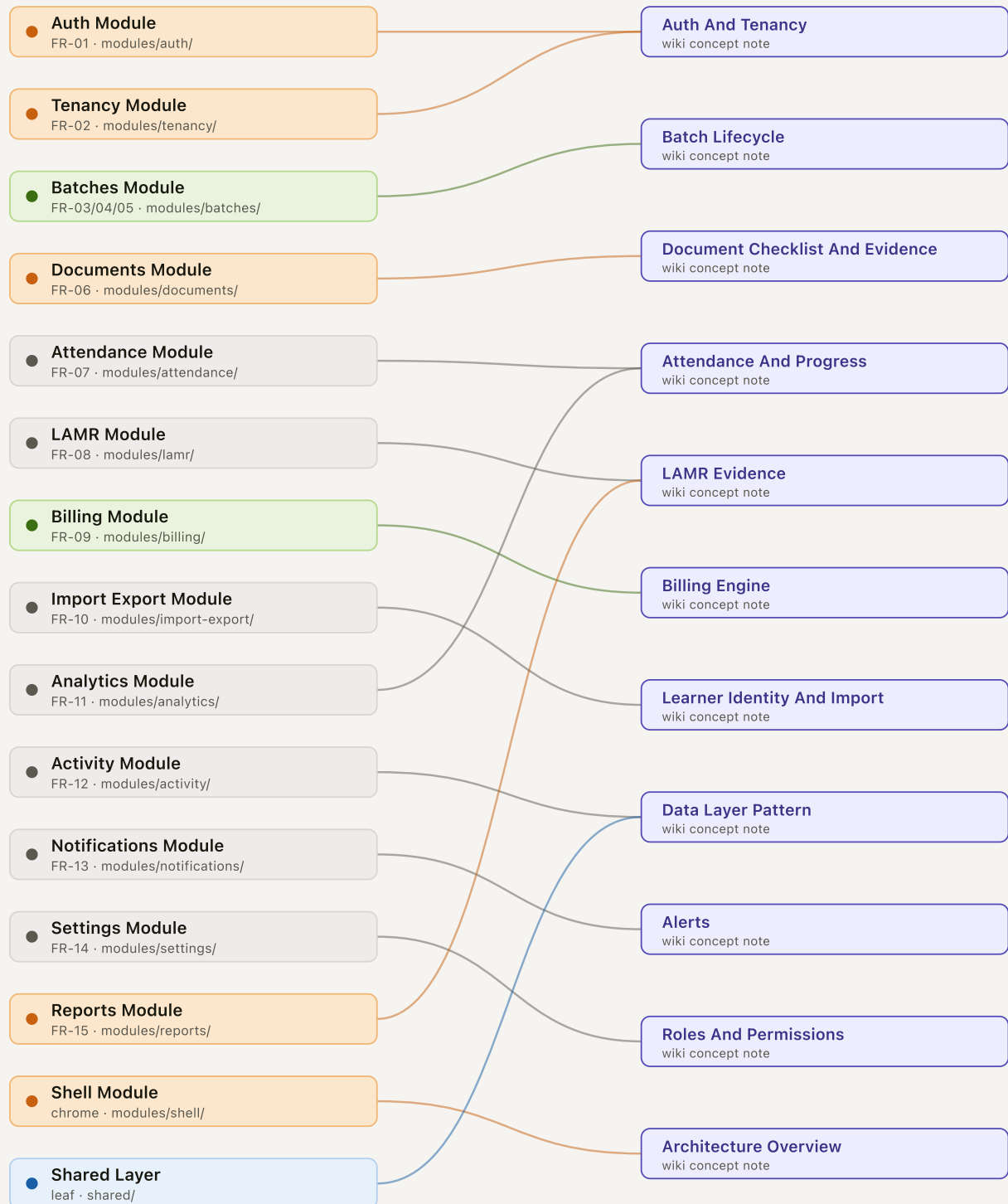
☐ Implemented ☐ Partial ☐ Stub ☐ Leaf

CODE — modules/<domain>

each box is a real graph node

CONCEPTS — wiki/

what each module implements



05 Module status — what is real today

Don't assume a prototype screen means the module is built. This is the ground truth from the `modules/` tree.

MODULE	FR	STATUS	CONCEPT NOTE
Auth Module Clerk identity + session resolution	FR-01	Partial	Auth And Tenancy
Tenancy Module Tenant-in-URL context; RLS scopes rows	FR-02	Partial	Auth And Tenancy
Batches Module Dashboard, lifecycle, urgency engine — reference impl	FR-03/04/05	Implemented	Batch Lifecycle
Documents Module Required docs, verification, evidence store	FR-06	Partial	Document Checklist And Evidence
Attendance Module Attendance-derived progress; 5-absence rule	FR-07	Stub	Attendance And Progress
LAMR Module Learners Achievement Monitoring Report	FR-08	Stub	LAMR Evidence
Billing Module Document-generating engine + statement preview	FR-09	Implemented	Billing Engine
Import Export Module Learner import wizard; ULI as key	FR-10	Stub	Learner Identity And Import
Analytics Module Aggregate metrics across batches	FR-11	Stub	Attendance And Progress
Activity Module activity_log audit trail	FR-12	Stub	Data Layer Pattern
Notifications Module In-app alerts, computed on read	FR-13	Stub	Alerts
Settings Module Tenant + user settings; role-gated	FR-14	Stub	Roles And Permissions
Reports Module Cross-batch reports	FR-15	Partial	LAMR Evidence
Shell Module App chrome: Sidebar, Topbar, MetricsRow	chrome	Partial	Architecture Overview
Shared Layer Props-only ui, domain types, mock seed	leaf	Leaf	Data Layer Pattern

06 Your guided reading path

Read these in order — each builds on the last. All live in the vault; open them in Obsidian or any editor.

NOTE	WHY NOW
1 CONTEXT.md	the ubiquitous language — read the glossary first so every term below lands
2 Product Overview	what the tool is, the official-systems boundary, tenants
3 Batch Lifecycle	AOU → NTP → TIP → Training → Assessment → Billing
4 Architecture Overview	Next.js → Supabase, Clerk, the RLS boundary
5 Data Layer Pattern	fetch → map → derive; the two type families — the pattern every module follows
6 Billing Engine	the document-generating engine (ADR-001) — the deepest domain area
7 Design System Rules	the non-negotiable visual constraints
8 Docs Precedence	which source wins when documents conflict

Entry points: Wiki Home (docs side) · Codebase Map (code side) · CLAUDE.md (agent/dev operating guide)

07 Rules that will trip you up

The non-negotiables. Most PR review comments trace back to one of these.

Design system (spec-mandated)

No emoji anywhere — icons are @tabler/icons-react line icons. IBM Plex fonts. Semantic color tokens only (never raw hex). Status by text + icon, never colour alone. WCAG 2.2 AA. Every data screen implements loading / empty / no-results / error / permission-denied / stale states, with a “Data as of” timestamp.

Role rules are load-bearing

Trainer-facing DTOs omit billing deadline, billing prep, NTP lag, BSRS and financial fields **server-side** — not CSS-hidden. Viewer is read-only, server-denied on writes. RLS is the boundary; UI hiding is usability only.

Docs precedence & billing

MASTER_PRD_SRS is the product source of truth, but **ADR-001 supersedes any “billing = preparation signal” wording**. The README's “billing preparation signal / 80% badge” language is the superseded model — billing is a document-generating engine (three tracks, tranches, readiness gate).

Static dirs are edit-blocked

assets/, preview/, screenshots/, ui_kits/, uploads/ are ported verbatim and blocked by a PreToolUse hook. Edit the source design files instead.

Migrations workflow

After any migration: regenerate database.types.ts → update affected mappers + domain types. Only module data/ layers may import database.types.

Owns the detail: CLAUDE.md (Design system rules, Role rules) · wiki/Design System Rules · wiki/Docs Precedence · docs/adr/ADR-001

08 Workflow: Linear, branches, PRs

- | | |
|---|---|
| → Linear team TESDA-CAMS (key TES) two-way syncs with GitHub. Create an issue on one side only — both sides pair automatically. | → Branches follow Linear's convention: klynejoshua13/tes-NN-... |
| → Skills /create-feature-branch and /create-pr automate the branch + PR flow with the Linear title. | → Migrations are additive; the base migration is canonical for schema + RLS. |

Owns the detail: CLAUDE.md (Git / workflow) · wiki/Project Process

09 Where things stand today

Foundation phase. Two modules are implemented end-to-end (**batches, billing**), several are partial (auth, tenancy, documents, reports, shell), and the rest are README stubs (see the status table in §05). Billing shipped its UI + on-screen statement preview (TES-70); real .docx generation and the append-only billing_records log remain. There is no test runner yet (Phase 0.4).

Honesty rule (ADR-002): the design prototype intentionally portrays the full ADR-001 target system even where the backing schema isn't migrated. So a polished screen is *not* evidence a module is implemented — check modules/<domain>/data/ for that.

Owns the detail: docs/IMPLEMENTATION_PLAN.md · docs/adr/ADR-001 · Linear TESDA-CAMS board

A Appendix — domain glossary

The quick reference. CONTEXT.md is the canonical, fuller version (including the three distinct Entrepreneurship concepts).

TERM	MEANS	AVOID
Batch	One RQM code's worth of scholars trained toward one qualification — the unit everything hangs off. One RQM code = one batch.	Class, cohort, group
RQM	The Request-for-QM authorization code naming a batch. Modeled as attributes <i>on the batch</i> (rqm_code, ntp_number, approved_slots...), never a separate table.	A PROGRAM_RQM table (rejected design)
NTP	Notice To Proceed — the approval authorizing a batch to run. Lives on the batch.	—
Approved slots	The billable-pax cap for a batch. You may bill for fewer scholars, never more.	—
ULI	The permanent national learner key (TESDA-issued). Same person at two schools = two tenant-scoped rows sharing one ULI.	Learner id, student number
Training progress	$\text{sessions_held} \div \text{total_sessions}$ (total = nominal_hours ÷ 8), snapshotted on the batch. Attendance-derived, never time-elapsed.	% complete, days elapsed
Ineligible scholar	A scholar with ≥5 absences , excluded from the allowance. The only eligibility rule (stored as max_absences=4; copy always says “≥5”).	attendance-% rule
Billing track	One of three parallel schedules a batch bills on — Training Cost, TSF/Allowance, Entrepreneurship — each on its own clock. Assessment Fee is out of scope.	“the billing” (there are three)
Billing tranche	A single releasable payment within a track, unlocked at an attendance threshold. Answers “which money is releasable,” not “how far through training.”	installment, milestone payment
Readiness gate	Per-tranche condition unlocking document generation: threshold reached AND that tranche's supporting docs all Verified. Tracked at tranche level, not per scholar.	“billing-ready” = threshold only
Billing document	A populated .docx generated from the school's own template. Append-only history — corrections re-generate a new versioned snapshot.	invoice, bill
→ Entrepreneurship component — a program-level trait (CFSP has it); the entire stage shows only for such programs. → Entrepreneurship billing — ₱800 × scholars enrolled in entrepreneurship who lack prior history. → Entrepreneurship history — per-scholar flag meaning <i>prior</i> entrepreneurship; one flag, three effects (–₱480 TSF, –3 days, excluded from ₱800 track). → Billing document — a populated .docx from the school's template; append-only versioned history.		